1. **Explain the process of building a max heap from an arbitrary array.**

   When building a max heap, we compare the values of a parent node with its children nodes. Starting at the bottom of the binary tree, we swap nodes if the *largest* child is larger than the parent.

   This process is repeated throughout the tree until no child node is larger than their parent node.

2. **Describe the Heapify() operation's functionality. When and how is it used in Heap Sort?**

   Heapify() goes through the previously iterated steps in order to build a max heap of a given unsorted array.

   Heapify() is used once for each index with a child node, recurring whenever necessary.

   When the first max heap is made, the first and last indexes swap and Heapify() is called. For the rest of the max heaps, however, the first index is the sole index used for Heapify(). This is because all of the other nodes are already sorted, so checking each node is redundant.

   Also, the size of the array is decreased by 1 for each iteration so that the sorted indexes are outside of the scope of Heapify().

3. **Derive and solve the recurrence relation for the Heapify() operation, clearly explaining each step.**

   For any heap, its height (h) will always be ≥ 1 and the maximum amount of nodes on the last level will always be 2^{h-1}.

Let L equal the number of left child nodes and R equal the number of right child nodes.

Therefore, $L = 2^h - 1$ and $R = 2^{h-1} - 1$. The number of nodes (n) in a given tree is $L + R + 1$ (accounting for the root node).

Therefore, the number of nodes is $2^h + 2^{h-1} - 1 = 3 \times 2^{h-1} - 1$.

Since a parent node only swaps with one child, we just need to find the ratio between a given group of child nodes and the total number of nodes.

For the let subtree, the ratio is $\frac{L}{n} = \frac{2^h-1}{3\times2^{h-1}-1}$. We then find the limit of the ratio, which is $\lim_{h\to\infty} \frac{2\times2^{h-1}-1}{3\times2^{h-1}-1} = \frac{2}{3}$. Since the limit is $\frac{2}{3}$, this means that the amount of children to be swapped with a parent is always $\leq \frac{2}{3}n$.

Now, we find how long it will take for the parent to be swapped with its children, which is O(1). To solve the recurrence relation, we add these two elements together.

So, we derive a recurrence relation of $T(n) = T(\frac{2}{3}n) + O(1)$.

In order to solve this recurrence relation, we use the recursion tree method, which involves plugging the relation into itself.

By substituting, we get $T(n) = T(\frac{4}{9}n) + O(1) + O(1) \rightarrow$ $T(n) = T(\frac{8}{27}n) + O(1) + O(1) \rightarrow T(n) = T(\frac{16}{81}n) + O(1) + O(1)$ $\rightarrow T(n) = T(n(\frac{2}{3})^k) + kO(1)$

The recurrence continues until a constant is reached, we can see when this happens by solving for k.

$n(\frac{2}{3})^k = 1 \rightarrow (\frac{2}{3})^k = \frac{1}{n} \rightarrow log((\frac{2}{3})^k) = log(\frac{1}{n}) \rightarrow$

$k \times log(\frac{2}{3}) = log(\frac{1}{n})$. Solving for k, we get $k = \frac{log(\frac{1}{n})}{log(\frac{2}{3})}$.

Plugging k back into the equation, we get

$T(n) = kO(1) = O(k) = O(\frac{log(\frac{1}{n})}{log(\frac{2}{3})})$.

Simplifying, we get a solved recurrence relation of $O(log\ n)$.

4. **What is the overall time complexity of Heap Sort? Explain how you derived your answer based on the recurrence relations and operations involved.**

   We previously found that the time complexity of Heapify() is $O(log\ n)$.

   When we call Heapify(), we recurse almost over every element.

   Because of this - n being the amount of elements in a given array - the overall complexity would be $T(n) = n \times O(log\ n)$.

   So, the overall time complexity of HeapSort() would be $O(nlog\ n)$.